

Cryptography Engineering Project 2 Report

Group 9

1 Task 1: Single Hash vs. Merkle Tree

1.1 Experimental Results

Test Environment Configuration

- Dataset size: 128 MB
- Number of blocks: $n = 2^7 = 128$

Single Hash (All-or-Nothing Approach)

- Resulting SHA-256:

```
0c05acf58bb36284a9e73f0222f2fd0621954bb8eb5c16ee50a45341b5cd4594
```

- Execution time: 0.088621 seconds

Merkle Tree Construction

- Merkle root:

```
3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
```

- Execution time: 0.137790 seconds
- Internal node memory overhead: 8255 bytes

```
PS C:\Users\88696\Downloads\project2> python -u "c:\Users\88696\Downloads\project2\task1.py"
=====
步驟 1: Single Hash (All-or-Nothing)
=====
Single Hash 結果: 0c05acf58bb36284a9e73f0222f2fd0621954bb8eb5c16ee50a45341b5cd4594
計算時間: 0.088621 秒

=====
步驟 2: Merkle Tree (2^7 Blocks)
=====
Merkle Root: 3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
建構時間: 0.137790 秒
內部節點記憶體開銷: 8255 Bytes
=====
```

Figure 1: Task 1 console output.

1.2 Discussion and Findings

Execution Time Comparison. The Single Hash method completed in approximately 0.088 seconds, while the Merkle Tree construction took 0.137 seconds. The Single Hash approach is inherently faster because it sequentially processes the continuous 128 MB data stream in a single pass. In contrast, the Merkle Tree incurs a slight time penalty. This delay is expected, as the algorithm must slice the file into 128 distinct blocks, compute 128 individual leaf hashes, and then execute iterative loops to concatenate and hash the internal nodes upward to the root.

Memory Overhead Analysis. To fulfill the requirement of analyzing the memory overhead of the tree's internal nodes, the measured overhead is 8255 bytes. Mathematically, a binary Merkle Tree with $n = 128$ leaf nodes requires $n - 1 = 127$ internal parent nodes. Storing 127 SHA-256 hash objects in Python aligns with this roughly 8 KB footprint.

2 Task 2: Efficient Error Localization

2.1 Implementation and Experiment Results

Top-down search algorithm. In order to find the corrupted block, the implementation compares the Merkle tree from the original correct data with the Merkle tree from the corrupted data. Starting from the root node, each traversal level needs only one child comparison because the experiment assumes that only one block is corrupted. If the left child hash does not match the trusted tree, the corruption is located in the left subtree, so the algorithm traverses down the left path. If the left child hash matches, the left subtree is intact and the corruption must be in the right subtree, so the algorithm traverses down the right path. This process continues until the affected leaf node is reached.

```
=====
Task 2: Efficient Error Localization
=====
1. Loading Trusted Merkle Tree...
2. Build Merkle Tree from corrupted data...
3. Locating the corrupted block...

[Output]
Trusted Merkle Root   : 3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Corrupted File Root  : 3f494602bb982a38d9ec097143ef997818209518e3d25f865cb20512473356ce
Corrupted Block Index: 67
Comparison Count      : 7
=====
```

Figure 2: Task 2 console output.

2.2 Discussion

In this experiment, the 128 MB file is divided into $n = 128$ blocks, which are used to construct a complete binary tree. The height H of this tree is $\log_2(128) = 7$.

Traversal Strategy. During the top-down traversal, because there is only a single bit error in the entire dataset, it is sufficient to check only one child node, specifically the left child, at each level to make a routing decision.

Comparison Count Proof. Each hash comparison eliminates half of the remaining search space. Traversing from the root at level 7 down to a leaf at level 0 requires descending exactly 7 levels. Because the implementation executes exactly one hash comparison per level, the total number of comparisons is exactly 7. This satisfies the theoretical constraint of $\log_2(n)$ comparisons.

3 Task 3: Efficient Node Replacement

3.1 Experimental Results

```
python3 task3.py

File generated: test_128mb.bin
Size: 128.0 MB
Block file generated: test_1mb.bin
Size: 1048576 bytes (1MB)

=====
Task 3: Efficient Node Replacement
=====

1. Loading original data...
2. Building original Merkle Tree...
Enter block index to replace (0 ~ 127): 67
3. Updating Merkle Tree with replace_node()...
4. Rebuilding full Merkle Tree for verification...

[Output]
Original Root           : 3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Updated Root (replace_node): c392a826e9a141f3f4dfef8baf007620a1b0d16e6fd888da5a3556b2d863039e
Updated Root (full tree)  : c392a826e9a141f3f4dfef8baf007620a1b0d16e6fd888da5a3556b2d863039e
Verification Result       : PASS
Path Update Time          : 0.000989 seconds
Full Reconstruction Time   : 0.078458 seconds
Performance Comparison     : 79.33x faster

=====
```

Figure 3: Task 3 console output.

3.2 Discussion and Findings

Task 3 evaluates whether the Merkle root can be updated efficiently after replacing one 1 MB block. Instead of rebuilding the entire Merkle Tree, `replace_node()` updates only the affected path from the modified leaf to the root.

Correctness Verification. Both update methods produced the same updated Merkle root.

```
c392a826e9a141f3f4dfef8baf007620a1b0d16e6fd888da5a3556b2d863039e
```

Since both methods produced the same root, the path update implementation is correct.

Hash Computation Comparison. The full reconstruction method recomputes the entire Merkle Tree, requiring 128 leaf hashes and 127 internal hashes, for a total of 255 hash computations. In contrast, `replace_node()` only recomputes the modified leaf and the 7 internal nodes on the path to the root, requiring only 8 hash computations. Therefore, the path update reduces the number of hash computations by:

$$\frac{255}{8} = 31.88$$

Runtime Comparison. The measured path update time was 0.000989 seconds, while the full reconstruction time was 0.078458 seconds.

$$\frac{0.078458}{0.000989} = 79.33$$

Therefore, the path update was approximately 79.33 times faster in this experiment. This demonstrates the main advantage of Merkle Trees for dynamic data. When only one block

changes, only the path from the modified leaf to the root needs to be updated, giving an $O(\log n)$ update cost instead of recomputing the entire file.

4 Task 4: Advanced Self-Healing

4.1 Implementation and Experiment Results

Task 4 integrates XOR parity recovery with Merkle Tree error detection. The input files are the corrupted 128 MB dataset, the trusted Merkle Tree generated from the original data, and the parity blocks generated from the original data. The clean original file is not used during repair.

The parity file stores one parity block for every pair of data blocks:

$$P_i = D_{2i} \oplus D_{2i+1}$$

After `locate_error()` identifies the corrupted block, the implementation selects the corresponding parity block and the sibling data block. If block D_j is corrupted, its sibling block D_s is still correct because the experiment contains only one bit flip in one block. The original block can therefore be reconstructed as:

$$D_j = P_{\lfloor j/2 \rfloor} \oplus D_s$$

The recovered block is then passed to `replace_node()`, which repairs only the affected leaf-to-root path in the corrupted Merkle Tree. The final verification compares the repaired root against the trusted root.

```
=====
Task 4: Advanced Self-Healing
=====
1. Loading corrupted data, trusted Merkle Tree, and parity blocks...
2. Building Merkle Tree from corrupted data...
3. Locating corrupted block using trusted Merkle Tree...
4. Recovering corrupted block with XOR parity and sibling block...
5. Repairing Merkle Tree with replace_node()...

[Output]
Trusted Merkle Root      : 3
    cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Corrupted File Root     :
    bb2b791c16712525185c86f30f39bc878365f4612423a106ad0f729f5003242b
Corrupted Block Index   : 42
Comparison Count        : 7
Parity Block Index Used : 21
Sibling Block Index      : 43
Repaired Merkle Root     : 3
    cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
Verification Result      : PASS
=====
```

4.2 Discussion and Findings

The corrupted block was identified as block 42 using only the corrupted file and the trusted Merkle Tree. Since $n = 128$, the tree height is $\log_2(128) = 7$, and the reported comparison count is exactly 7. This confirms that the detection step keeps the required $O(\log n)$ localization behavior.

The parity index used for recovery was 21 because block 42 belongs to the block pair (42, 43), and $\lfloor 42/2 \rfloor = 21$. Since block 42 is the even member of the pair, block 43 is the corresponding sibling block. The repair process computes:

$$D_{42} = P_{21} \oplus D_{43}$$

The repaired Merkle root is identical to the trusted Merkle root:

```
3cd74d8d7171cce47fb900a1aca739b74286b6f84e5da9490f54559dbecc8668
```

Therefore, the corrupted block can be both located and recovered without access to the original clean 128 MB file. The Merkle Tree provides efficient localization, while XOR parity provides the missing data needed for correction. Together, they implement a self-healing mechanism for a single corrupted block.

5 Summary Table

Operation	Single Hash	Merkle Tree	Improvement Factor
Detecting error	Scans 128 MB	Checks the trusted root against the corrupted root	Root mismatch gives immediate detection after root computation
Localizing error	Impossible	7 comparisons	$\log_2(128)$ search
Updating 1 block	Computes 128 MB	Computes 1 MB block hash plus 7 internal hashes	31.88x fewer hashes
Proof size	No local proof	$7 \times 32 = 224$ bytes	Logarithmic proof size

Table 1: Comparison summary for $n = 2^7$ blocks.